

Travelo Backend

NestJS Architecture Guide

Architecture | Design Patterns | Module System
Data Models | Routing | Data Flow

Tech Stack: NestJS 12 | Drizzle ORM | PostgreSQL 17 | TypeScript

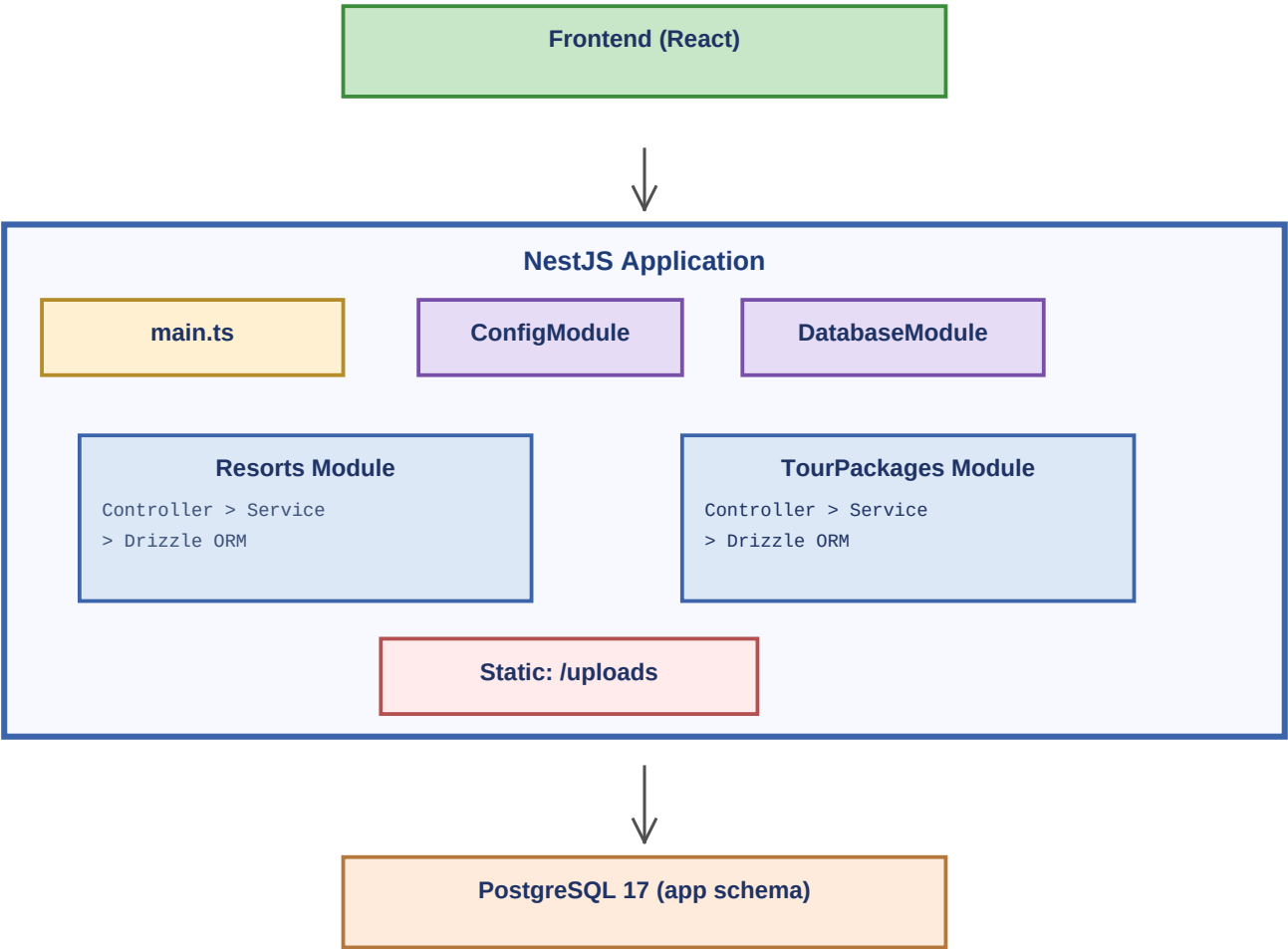
Table of Contents

1. High-Level Architecture	3
2. Module System & Dependency Injection	4
3. Request Lifecycle	5
4. Database Schema & Models	6
5. Feature Module Deep Dive	7
6. Routing Map	8
7. Data Flow Diagrams	9
8. File Structure Reference	10

1. High-Level Architecture

The Travelo backend follows a layered, modular architecture powered by NestJS. Each feature is encapsulated in its own module with a clear separation between HTTP handling (Controller), business logic (Service), and data access (Drizzle ORM).

Figure 1: System Architecture Overview



Key principle: Modules are isolated. A feature module only knows about its own Controller and Service. The DatabaseModule is @Global, so every service can inject the DRIZZLE token without explicitly importing the database module.

2. Module System & Dependency Injection

What is a Module?

A module is a class decorated with `@Module()` that groups related components. NestJS builds a dependency graph from all modules and resolves dependencies automatically.

The Three Types of Modules

Feature Modules (ResortsModule, TourPackagesModule)

Own a specific domain feature. Contains Controller (HTTP) + Service (logic). Each feature is self-contained and can be developed/tested independently.

Global Modules (DatabaseModule)

Decorated with `@Global()`. Their exports are available to ALL modules without explicit import. Used for shared infrastructure like database connections.

Root Module (AppModule)

The entry module imported by `main.ts`. Wires all feature modules together. Also imports `ConfigModule.forRoot()` for `.env` loading.

Dependency Injection Flow

Figure 2: How DI resolves the DRIZZLE token

```

main.ts
|
v
AppModule
  imports: [ConfigModule, DatabaseModule, ResortsModule]
      |
      v
      DatabaseModule (@Global)
        providers: [{
          provide: "DRIZZLE",
          useFactory: (ConfigService) => ...
        }]
          |
          Inject("DRIZZLE") available everywhere
            |
            +-----+-----+
            |               |
            v               v
ResortsService      TourPackagesService
constructor(        constructor(
  @Inject("DRIZZLE")  @Inject("DRIZZLE")
  private db          private db
)                    )

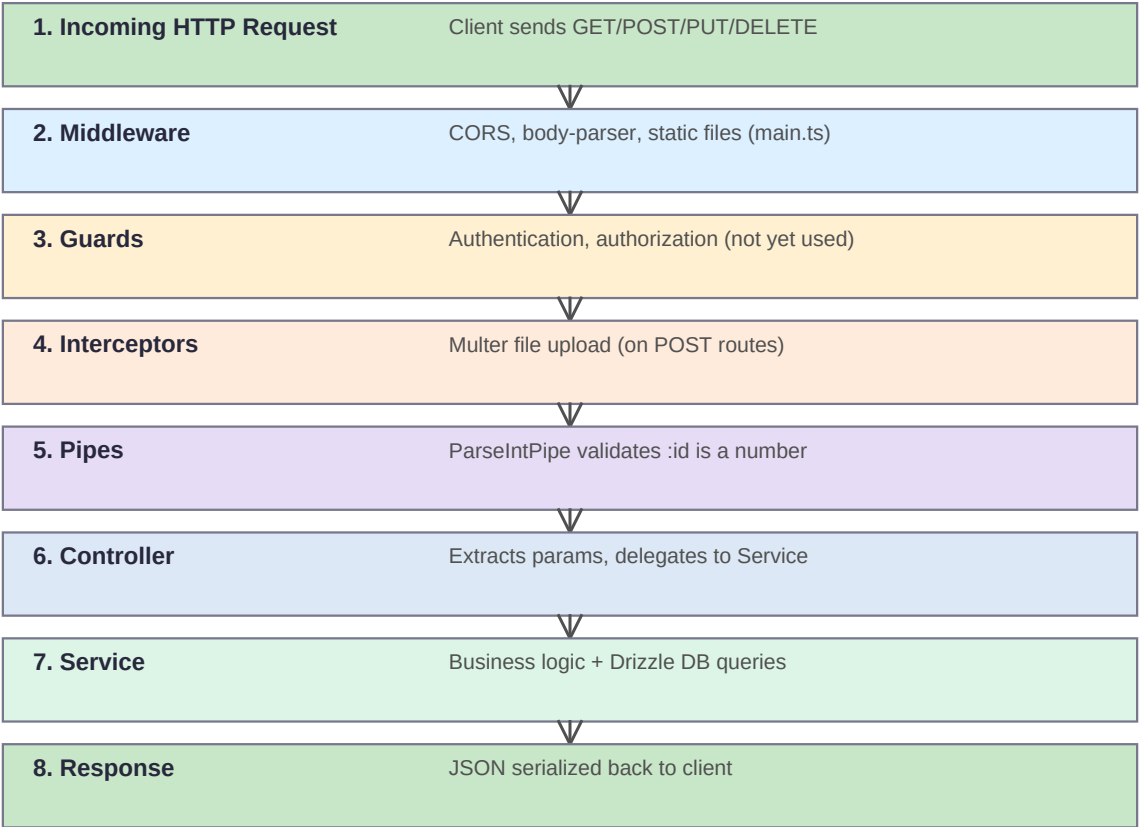
```

The `@Inject("DRIZZLE")` decorator tells NestJS: "find the provider registered with the token DRIZZLE and pass it to my constructor." Because DatabaseModule is `@Global`, this works from any module without importing it.

3. Request Lifecycle

Every HTTP request follows this pipeline. Understanding this flow is key to knowing where to add validation, transformation, and error handling.

Figure 3: Request Lifecycle Pipeline



Concrete Example: GET /api/resorts?page=2

1. Client sends: GET /api/resorts?page=2
2. CORS middleware allows the request
3. No guards (no auth configured)
4. No interceptors (GET, no file upload)
5. No pipes needed (no :id param)
6. ResortsController.findAll("2")
 calls this.resortsService.findAll(2, 12)
7. ResortsService runs:
 SELECT count(*) FROM app.resorts
 SELECT * FROM app.resorts
 ORDER BY created_at DESC
 LIMIT 12 OFFSET 12
8. Returns: { success: true, data: [...],
 pagination: { page: 2, ... } }

4. Database Schema & Models

All tables live in the "app" PostgreSQL schema, defined in `src/db/schema.ts` using Drizzle ORM. Drizzle generates type-safe queries from these definitions.

Resorts Table

Table: `app.resorts`

Column	Type	Constraint
<code>id</code>	<code>SERIAL</code>	<code>PRIMARY KEY</code>
<code>location</code>	<code>TEXT</code>	<code>NOT NULL</code>
<code>description</code>	<code>TEXT</code>	<code>nullable</code>
<code>price</code>	<code>NUMERIC(10, 2)</code>	<code>DEFAULT 0</code>
<code>ratings</code>	<code>NUMERIC(3, 1)</code>	<code>DEFAULT 0</code>
<code>image1</code>	<code>TEXT</code>	<code>nullable</code>
<code>image2</code>	<code>TEXT</code>	<code>nullable</code>
<code>image3</code>	<code>TEXT</code>	<code>nullable</code>
<code>created_at</code>	<code>TIMESTAMP</code>	<code>DEFAULT NOW()</code>

Tour Packages Table

Table: `app.tour_packages`

Column	Type	Constraint
<code>id</code>	<code>SERIAL</code>	<code>PRIMARY KEY</code>
<code>location</code>	<code>TEXT</code>	<code>NOT NULL</code>
<code>description</code>	<code>TEXT</code>	<code>nullable</code>
<code>total_days</code>	<code>INTEGER</code>	<code>DEFAULT 1</code>
<code>price</code>	<code>NUMERIC(10, 2)</code>	<code>DEFAULT 0</code>
<code>ratings</code>	<code>NUMERIC(3, 1)</code>	<code>DEFAULT 0</code>
<code>category</code>	<code>TEXT</code>	<code>DEFAULT 'Kerala'</code>
<code>image1</code>	<code>TEXT</code>	<code>nullable</code>
<code>image2</code>	<code>TEXT</code>	<code>nullable</code>
<code>image3</code>	<code>TEXT</code>	<code>nullable</code>
<code>created_at</code>	<code>TIMESTAMP</code>	<code>DEFAULT NOW()</code>

Category values are validated in the service layer: `Kerala` | `International` | `NorthEast`. Images are stored as text paths (e.g., `/uploads/1234567890-abc.jpg`).

5. Feature Module Deep Dive

The Module-Controller-Service Pattern

Every feature module follows the same 3-file structure. Here is the anatomy of the Resorts module:

Figure 4: Resorts Module Internal Structure

```
resorts/
  resorts.module.ts      <-- Wiring
    @Module({
      controllers: [ResortsController],
      providers:    [ResortsService],
      exports:      [ResortsService],
    })

  resorts.controller.ts  <-- HTTP Layer
    @Controller("api/resorts")
    @Get()              findAll()
    @Get(":id")          findOne(id)
    @Post()              create(files, body)
    @Put(":id")          update(id, body)
    @Delete(":id")       remove(id)

  resorts.service.ts     <-- Business Logic
    @Injectable()
    findAll(page, limit) -> paginated query
    findOne(id)          -> single record
    create(data)          -> INSERT RETURNING
    update(id, data)      -> UPDATE RETURNING
    remove(id)           -> DELETE RETURNING
```

Responsibility Split

Controller handles:

Route definitions (@Get, @Post, etc.) | Parameter extraction (@Param, @Query, @Body) | File upload middleware (@UseInterceptors) | Type conversion (ParseIntPipe) | Delegation to service

Service handles:

Database queries via Drizzle ORM | Business validation (category checks) | Error handling (NotFoundException, BadRequestException) | Data transformation | Pagination logic

Rule of thumb: if it involves HTTP (routes, params, files), it goes in the Controller. If it involves data or logic, it goes in the Service.

6. Routing Map

All API routes are prefixed with /api. Here is the complete route table:

Resorts API

Method	Route	Description
GET	/api/resorts	List all (paginated)
GET	/api/resorts/:id	Get one by ID
POST	/api/resorts	Create (multipart)
PUT	/api/resorts/:id	Update fields
DELETE	/api/resorts/:id	Delete by ID

Tour Packages API

Method	Route	Description
GET	/api/tour-packages	List all
GET	/api/tour-packages/category/:cat	Filter by category
GET	/api/tour-packages/:id	Get one by ID
POST	/api/tour-packages	Create (multipart)
PUT	/api/tour-packages/:id	Update fields
DELETE	/api/tour-packages/:id	Delete by ID

Root API

GET	/	Health check (returns "Hello World!")
-----	---	---------------------------------------

Static Files

GET /uploads/{filename} serves static files from the uploads/ directory. Used to access uploaded resort and tour package images.

7. Data Flow Diagrams

Read Flow: GET /api/resorts/:id

Figure 5: Read Request Flow

```
Client
|
| GET /api/resorts/42
v
ResortsController.findOne(42)
| @Param("id", ParseIntPipe) => 42
v
ResortsService.findOne(42)
|
| db.select().from(resorts)
| .where(eq(resorts.id, 42))
v
PostgreSQL: SELECT * FROM app.resorts WHERE id=42
|
| Found? -> return record as JSON
| Not found? -> throw NotFoundException (404)
v
Client receives JSON response
```

Write Flow: POST /api/tour-packages (with images)

Figure 6: Create Request with File Upload

```
Client (multipart/form-data)
|
| POST /api/tour-packages
| Body: { location, description, price, ... }
| Files: [image1.jpg, image2.png]
v
Multer Interceptor
| Saves files to uploads/ with unique names
| Rejects: >5MB, non-JPEG/PNG
v
TourPackagesController.create()
| Maps files -> ["/uploads/abc.jpg", ...]
v
TourPackagesService.create(data)
| Validates category (Kerala/Intl/NorthEast)
| INSERT INTO app.tour_packages (...)
| VALUES (...) RETURNING *
v
Returns created record with all fields
```

8. File Structure Reference

Complete project file tree with the role of each file:

src/main.ts	Application entry point. Creates NestExpressApplication, enables CORS, mounts static assets, starts listening.
src/app.module.ts	Root module. Imports ConfigModule, DatabaseModule, ResortsModule, TourPackagesModule.
src/app.controller.ts	Root controller. Single GET / route for health check.
src/app.service.ts	Root service. Returns "Hello World!" for health check.
src/db/schema.ts	Drizzle table definitions. Exports resorts and tourPackages tables under the "app" pg schema.
src/db/database.module.ts	@Global module. Creates pg.Pool + Drizzle instance, exports as "DRIZZLE" token.
src/resorts/resorts.module.ts	Feature module. Wires ResortsController + ResortsService.
src/resorts/resorts.controller.ts	HTTP layer. GET/POST/PUT/DELETE routes with Multer file upload on POST.
src/resorts/resorts.service.ts	Business logic. Drizzle queries, pagination, NotFoundException on missing records.
src/tour-packages/tour-packages.module.ts	Feature module. Wires TourPackagesController + TourPackagesService.
src/tour-packages/tour-packages.controller.ts	HTTP layer. Same CRUD + category filter route + Multer (max 3 images).
src/tour-packages/tour-packages.service.ts	Business logic. Same as resorts + category validation (BadRequestException).
uploads/	Static directory for uploaded images. Served via /uploads URL prefix.
drizzle/	Auto-generated migration files from drizzle-kit.
drizzle.config.ts	Drizzle Kit configuration (DB URL, schema path, output dir).
.env	Environment variables: DATABASE_URL, PORT.